SAGAR INSTITUTE OF SCIENCE & TECHNOLOGY
Approved by AICTE, New Delhi & Govt. of MP. Affiliated to RGPV & BU, Bhopal

Department of Computer Science and Engineering (CSE)
Gandhi Nagar, Bhopal, Madhya Pradesh

Cluster Kit

Supercomputing, built from lab desktops

A small parallel computer made from the PCs in our lab

Submitted by

Name	Enrolment No.	Branch
Divyansh Shukla	0187AS251047	CSE – Artificial Intelligence & Data Science
Vanshika Wadhwa	0187CS251288	Computer Science & Engineering
Abhinav Dhoke	0187CS251008	Computer Science & Engineering
Sneha Rajpoot	0187CS253D09	Computer Science & Engineering
Ankush Kumar	0187CS241036	Computer Science & Engineering

CSE: Computer Science and Engineering

Under the guidance of

Mr. Nargish Gupta
Head of Department, Computer Science and Engineering

Prof. Abizer Hakimuddin
Assistant Professor
Department of Computer Science and Engineering

Sagar Institute of Science and Technology (SISTec), Bhopal
October 2026

Abstract

Supercomputers are fast not because of one enormous processor but because thousands of ordinary computers, joined by a network, work on pieces of the same problem [1, 3]. Students rarely get to use such machines, yet every college lab holds rows of capable desktops that sit idle for most of the day. This paper asks how much of the supercomputing idea survives at lab scale, with no special hardware and no setup expertise. We present Cluster Kit, a small program that turns a group of lab PCs into one cluster. One PC acts as the head: it splits a job into chunks, hands them to the other PCs, which compute on all of their cores, and collects the results. The PCs find each other on the network automatically, so nothing needs to be configured. We evaluate Cluster Kit on four identical Ubuntu PCs with a SHA-256 [5] hash search, a job that splits perfectly and whose correct answer is known in advance. Four PCs ran 3.86 times faster than one, an efficiency of 96.4%, in line with the near-linear speed-up predicted by Amdahl's [1] and Gustafson's [2] laws. A Python program that took 36.98 seconds on a Windows laptop finished in 2.165 seconds on the cluster. We compare the design with MPI [3] and Dask [4] and release the source code [6]. The results show that a usable parallel computer can be assembled from the machines a college already owns. Because the same method applies to any number of PCs, a lab of several hundred machines used at full capacity could process very large data sets far faster than any single computer, although we measured only four.

Keywords: parallel computing, cluster, supercomputer, speed-up, SHA-256, commodity hardware

Contents

1 Introduction 2
2 Background 2
3 How Cluster Kit works 2
4 Test setup 4
5 Results 5
6 Comparison with MPI 7
7 Limitations and future work 7
8 Conclusion 8

1 Introduction

A supercomputer is not one giant processor. It is thousands of ordinary computers connected by a fast network, all working on pieces of the same problem. We wondered whether the same idea works on a much smaller scale, with the desktops that already sit in our lab.

Our goal was simple. We wanted to connect a few lab PCs with as little setup as possible, run one job across all of them, and measure honestly how much time it saves. We also wanted the result to be something other students could run themselves: copy one program to each PC, start it, and the PCs should find each other.

A supercomputer can process very large data sets quickly because it spreads them over many machines at once. A college has the same ingredient in bulk: if all 500 computers in its labs were used at full capacity, a large task could be finished in a fraction of the time one machine needs. Cluster Kit is the technology we built to make that possible. We tested it on four PCs, so the 500-PC case is what the design aims at, not something we measured.

This report describes how Cluster Kit works, how we tested it in the lab, and what we found. Section 5 has the measurements; Section 6 compares our design with MPI, the tool real supercomputers use.

2 Background

High-performance computing (HPC) is about solving large problems faster by using many processors at once. A *cluster* is a group of computers on a network that behave like one bigger machine. Usually one computer, the *head*, hands out work and collects the answers, and the others, the *workers*, do the computing.

Two numbers describe how well a cluster performs. The *speed-up* is how many times faster a job runs on p computers than on one. The *efficiency* is the speed-up divided by p; at 100% no time is lost to coordination. Amdahl [1] and Gustafson [2] showed that the speed-up depends on how much of the work can be done in parallel. Our test job can be split perfectly, so in theory four PCs should be close to four times faster.

For the test we used SHA-256 [5], a cryptographic hash function. Our program hashes every number from 1 to n and counts the hashes that start with a few zeros. Each number can be checked on its own, so the job splits cleanly into chunks, and because the expected number of matches is known in advance, it is easy to check that the cluster gave the right answer.

3 How Cluster Kit works

Cluster Kit is a single program, `ck`, written in Rust. The same program runs as the head, as a worker, or as the client that submits a job. Figure 1 shows how the parts connect in our lab setup.

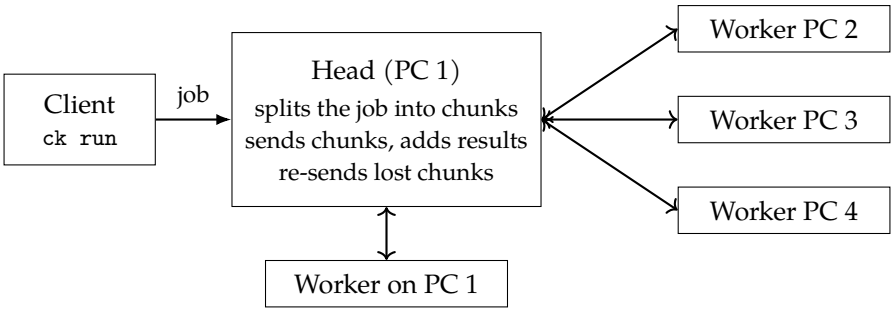


Figure 1: The head splits the job and shares it between three worker PCs and a worker running on the head PC itself.

When a job starts, the head cuts the range of numbers into chunks. Instead of dividing the work up front, it lets each worker ask for a chunk whenever it is free. A worker always keeps one extra chunk queued, so while it computes one chunk the next is already waiting and it never sits idle waiting for the network. A side effect is that faster PCs simply ask more often and end up doing more of the work, with no tuning on our part. Inside each PC the chunk is split again across all CPU cores using the Rayon library.

If a worker is unplugged or crashes in the middle of a job, the head notices the closed connection and gives that worker's unfinished chunks to the others, so the job still finishes. The head also serves a small web dashboard that shows each worker, its CPU use and the chunks it has completed.

3.1 Networking

We use two protocols for two different jobs (Table 1). To find each other, the head sends a small UDP broadcast every few moments saying "I am the head". A worker started anywhere on the same network hears it and connects, so nobody has to type IP addresses. If one broadcast is lost it does not matter, because the next one follows shortly. The actual work and results travel over TCP instead, because every chunk and every result must arrive and nothing can be lost. Each worker keeps one TCP connection open for the whole job.

Table 1: Network ports used by Cluster Kit

Protocol	Port	Purpose
UDP	7701	head announces itself (discovery)
TCP	7700	chunks of work and results
HTTP	7702	live dashboard in a browser

Figure 2 shows the messages exchanged between the head and one worker during a job.

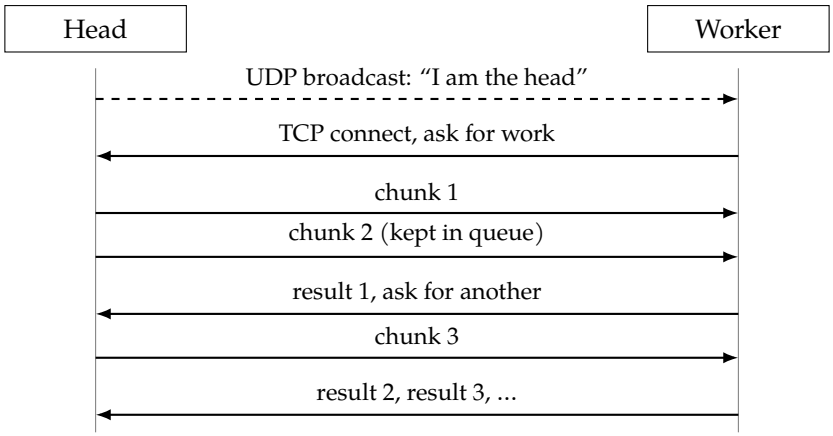


Figure 2: Messages between the head and one worker. The dashed arrow is UDP; the rest go over one TCP connection.

3.2 Other parts of the project

Jobs do not have to be written in Rust. Our test program, `simple_sha256.py`, is plain Python, and the engine can run it on every worker; the head can even send the script to workers that do not have a copy, so no shared folder is needed. The repository also contains a second engine based on Dask [4] for Python users. The engine itself is one portable program that runs on Ubuntu, Windows and macOS.

4 Test setup

We ran all tests in our lab on four identical desktops running Ubuntu 26.04.1, each with 12 CPU threads and 7.1 GB of RAM, connected by wired Ethernet (round-trip time under 1 ms). We labelled the head PC “Master” with a sticky note so we would not mix them up (Figure 3). We built `ck` once on the head and copied the program to the three workers.

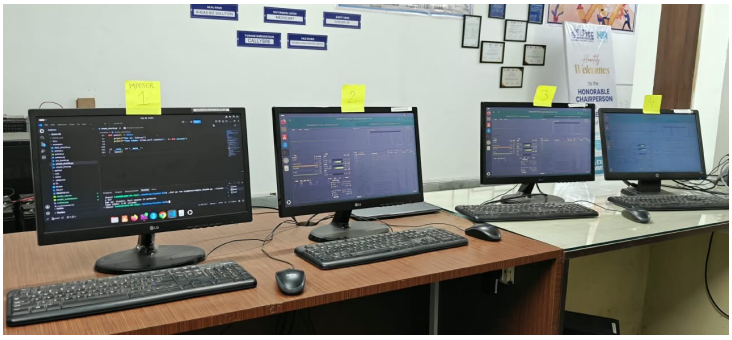


Figure 3: The four PCs we used. PC 1, marked “Master”, is the head.

For the scaling test (Section 5.2) we measured seven setups: each PC on its own, then 2, 3 and 4 PCs. To keep the comparison fair we ran every setup seven times, rotating through the setups after one warm-up run, so that any slow drift such as the PCs heating up would affect all of them equally. We report the median of the seven runs.

5 Results

5.1 Windows laptop and the cluster

Before the controlled test we ran our Python example on a normal Windows laptop and then on the four-PC cluster. Both runs found the same 766 matching hashes, so they did exactly the same work. The laptop took 36.98 seconds and the cluster took 2.165 seconds, 17 times faster (Figure 4, Table 2).

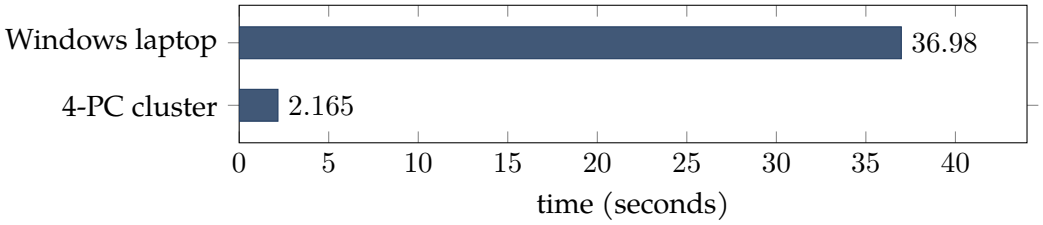


Figure 4: Time taken by the same program on a Windows laptop and on the cluster.

Table 2: Windows laptop compared with the cluster

	Windows laptop	4-PC cluster	Difference
Time	36.98 s	2.165 s	34.815 s saved
Matching hashes	766	766	same
Speed-up	1	17.08	94.1% less time

This number needs some care. The laptop is a different machine, the run was done only once, and we did not record whether it used one core or all of them, while the cluster had 48 threads. So the 17 times figure shows the practical gain over a laptop, not what clustering alone adds. For that we did the controlled test below. During our live demo in the lab the cluster ran the same program on 50 million numbers in 3.87 seconds (Figure 7).

5.2 Controlled scaling test

Here we kept everything the same (the PCs, the program and the amount of work per PC) and changed only the number of PCs. Table 3 and Figure 5 show the results.

Table 3: Scaling results, median of seven runs

PCs	M hashes/s	Speed-up	Efficiency	Time for 10^{11} hashes
1	20.55	0.91	100.0%	81 min
2	44.86	1.98	102.1%	37 min
3	67.48	2.98	100.3%	25 min
4	87.38	3.86	96.4%	19 min

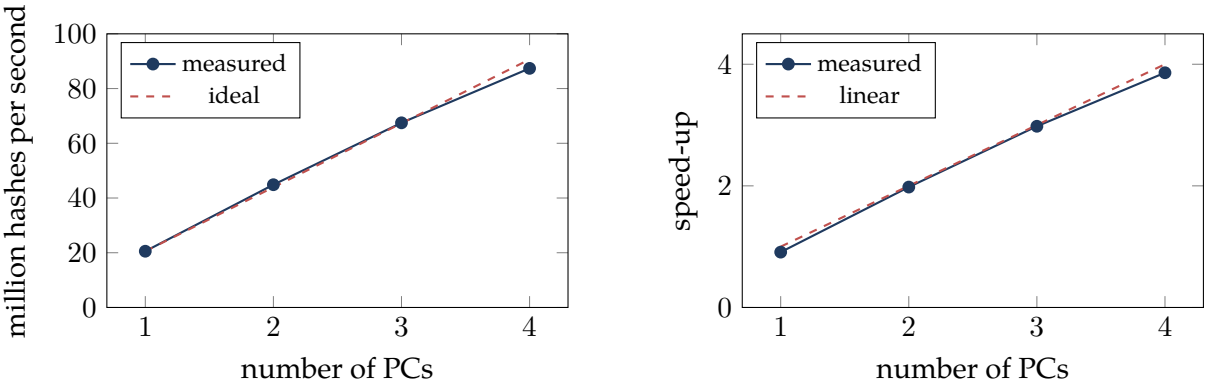


Figure 5: Left: hashing speed against the ideal, which is the sum of each PC's own speed. Right: speed-up against perfect linear scaling.

With all four PCs the cluster hashed 87.38 million numbers per second, 3.86 times the speed of an average single PC and 96.4% of what the four PCs manage separately. Less than 4% of the time went into coordination. In practical terms, a job that would take 81 minutes on one of our lab PCs takes about 19 minutes on four. Each added PC gave a clear, statistically significant improvement (permutation test, $p = 0.0003$), and every repeated run returned the same number of matches.

We noticed one oddity. On its own, PC 1 was about 12% slower than the other three, even though the machines are identical. It was the head and was also running another of our services at the time, which most likely took some of its CPU. Because of this we calculate the speed-up against the average single PC (22.66 million hashes per second) rather than against PC 1 alone, which would have inflated the result to 4.25.

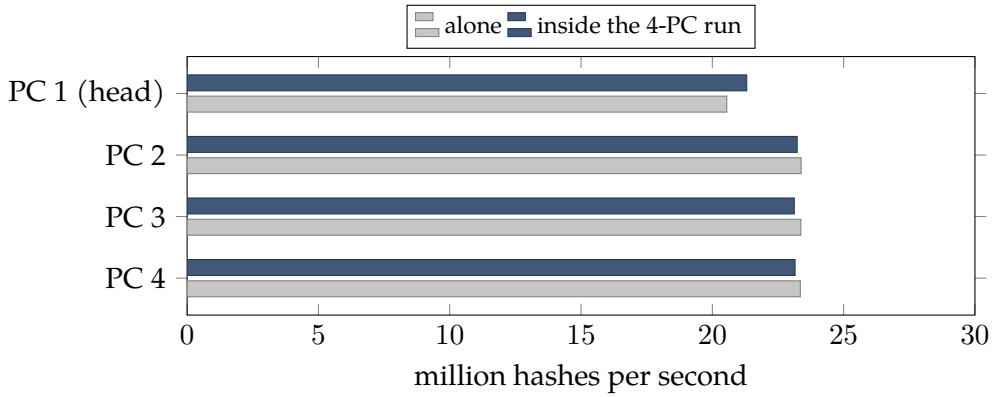


Figure 6: Each PC's speed on its own and while working inside the four-PC cluster.

Figure 6 shows that each PC kept almost all of its own speed while working in the cluster. The work also split nearly evenly, with each PC doing between 23.9% and 25.7% of the numbers, even though we never told the head how to divide it. PC 1 did the smallest share, which fits with it being the slowest. Every PC spent 96–99% of the job actually computing.

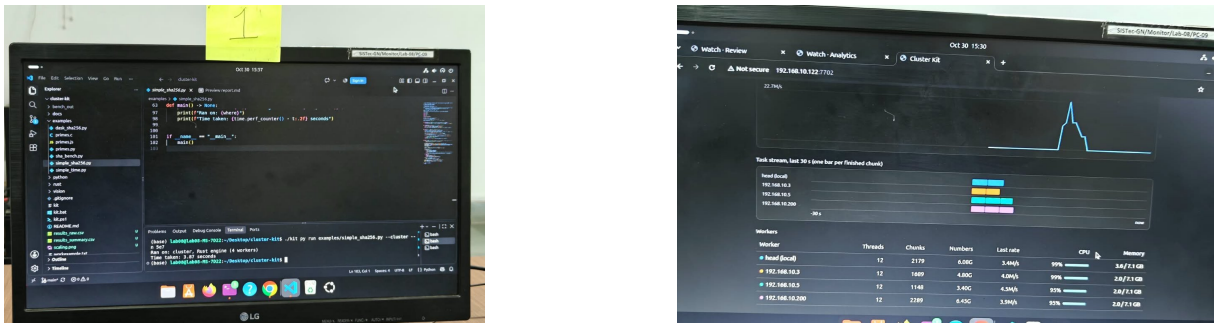


Figure 7: From our lab demo. Left: the head PC after running `simple_sha256.py` on the cluster ("Ran on: cluster, Rust engine (4 workers)", 3.87 s). Right: the dashboard during a run, with all four PCs at 95–99% CPU.

6 Comparison with MPI

Real supercomputers mostly use MPI [3], the Message Passing Interface. MPI is built for the highest possible speed on dedicated hardware and supports any pattern of communication between processes. Cluster Kit aims at something different: the least possible setup on ordinary lab PCs. We did not benchmark MPI ourselves, so Table 4 compares only how the two are set up and how they behave.

Table 4: Cluster Kit and a typical MPI setup

	Typical MPI setup	Cluster Kit
Finding the PCs	IP addresses listed in a host file	automatic, using UDP broadcast
Starting a job	SSH keys and `mpirun` on every PC	start the program on each PC
Dividing the work	usually fixed in advance	each PC asks for work when it is free
PCs of different speed	the slowest one sets the pace	faster PCs do more
If a PC fails	the job usually stops	its chunks go to the other PCs
Communication	any pattern between processes	only independent chunks

For our kind of job, where every chunk is independent, we expect MPI would be about as fast, since there is less than 4% left to gain. Where Cluster Kit is clearly easier is the setup and the way it copes with a PC dropping out. Measuring an MPI version on the same four PCs would be a good next step.

7 Limitations and future work

Cluster Kit only helps with work that can be cut into independent pieces; programs whose parts need to talk to each other constantly would need something like MPI. All PCs also have to be on the same local network, and the head PC coordinates every job, so if the head goes down the job stops. Our tests used only four PCs and short runs of about three seconds each.

There are several things we would like to try next: running on more than four PCs to see where the head becomes a bottleneck, adding GPU workers, adding authentication and encryp-

tion, measuring an MPI version for a direct comparison, and running real scientific or data-processing jobs instead of a hash search.

8 Conclusion

We set out to see whether the lab PCs we use every day could work together like a small supercomputer. With Cluster Kit, four of them ran our test job 3.86 times faster than one PC, at 96.4% efficiency, and the same program finished 17 times faster than on a Windows laptop. The PCs find each other on their own, the work spreads evenly without any tuning, and the job survives a PC dropping out. The same method should carry over to a whole lab, and to the 500 computers a college may have, which is the next thing we want to measure. For work that splits into independent pieces, a handful of ordinary desktops gets surprisingly close to the ideal.

References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[2] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[3] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.0*, 2021.

[4] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conference*, 2015, pp. 130–136.

[5] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[6] Cluster Kit source code, `https://github.com/TheDivyanshShukla/cluster-kit`.